

Dead Letter Queue (DLQ)

1. Overview

A **Dead Letter Queue (DLQ)** is a queue used to store messages, requests, or events that **could not be processed successfully** after one or more retries.

Instead of losing failed requests or blocking the main system, we isolate them for:

- Later retry
- Manual inspection
- Debugging
- Reprocessing
- Alerting

DLQ is widely used in:

- Message queues
- Kafka pipelines
- Payment systems
- Third-party API integrations
- Email/SMS delivery systems
- Order processing systems

2. Why Dead Letter Queue is Needed

Suppose your system calls a third-party API:

None

Order Service → Shipping API

If API fails due to:

- Timeout
- Rate limiting
- Temporary outage
- Invalid payload
- Network issue

Without DLQ:

None

Request fails

Data lost

User impacted

With DLQ:

None

Request fails

↓

Store failed request in DLQ

↓

Retry later / inspect later

3. Core Idea

When processing fails:

1. Try normal retries first
2. If still failing
3. Move request/message to DLQ

This prevents poison messages from repeatedly breaking the system.

4. Example Flow

None

Receive Payment Event

↓

Call Fraud Detection API

↓

Fails 3 times

↓

Move event to DLQ

↓

Continue processing other events

5. Real-World Example: Email Service

Suppose sending password reset email.

Failure reasons:

- SMTP outage
- Invalid email server
- Temporary network issue

System behavior:

None

Send email fails

↓

Put request in DLQ

↓

Retry after 10 mins

User request isn't lost.

6. Why DLQ is Important

Without DLQ, one bad message can:

- Be lost permanently
- Block queue consumers
- Cause infinite retries
- Slow down entire pipeline
- Hide production issues

With DLQ:

Main system continues

Failures isolated

Better observability

Safer retries

7. Where to Store DLQ? (Trade-offs)

Option 1: Drop Failed Requests

Simplest approach.

None

Failure → Discard request

Use only when missing request is acceptable.

Examples:

- Analytics event
- Low-value logs
- Non-critical telemetry

Pros

- Very simple
- No infrastructure cost

Cons

- Permanent data loss

Option 2: Local In-Memory / Local Queue

Use try-catch and store locally.

None

Failure → Save locally → Retry later

Pros

- Easy implementation
- Fast

Cons

- Lost if machine crashes
- Hard to scale multiple instances

Option 3: Persistent Queue / Broker (Best Production Choice)

Use:

- Kafka
- RabbitMQ
- AWS SQS DLQ
- Google Pub/Sub DLQ

None

Failure → Push to Kafka DLQ topic

Pros

- Durable
- Scalable
- Retry workers can consume later
- Centralized monitoring

Cons

- More complexity
- Infrastructure cost

8. DLQ Architecture

None

Main Queue



Consumer



Success → Done

Failure after retries



Dead Letter Queue



Reprocessor / Manual Fix / Alert

9. Retry Strategy Before DLQ

Usually don't send immediately to DLQ.

Use retries first:

- Immediate retry
- Exponential backoff
- Retry 3–5 times

Then DLQ.

Example:

None

Try 1 fail

Try 2 fail

Try 3 fail

→ DLQ

10. Common Reasons Messages Reach DLQ

Temporary Errors

- API down
- Timeout
- DB unavailable

Permanent Errors

- Invalid schema
- Missing required field
- Bad data format
- Deleted user account

Temporary errors can often be retried. Permanent errors may need fixing.

11. Poison Messages

A **poison message** is one that always fails processing.

Example:

JSON

```
{ "price": "abc" }
```

If consumer expects number.

Without DLQ:

None

Consumer retries forever

Queue blocked

With DLQ:

None

Move bad message to DLQ

Other messages continue

12. Kafka DLQ Example

Use separate topic:

None

orders-topic

orders-dlq-topic

payments-dlq-topic

Consumer writes failed messages to DLQ topic with:

- Original payload
- Error reason
- Timestamp
- Retry count

13. What to Store in DLQ Message

Include metadata:

None

Original payload

Failure reason

Timestamp

Retry count

Source topic

Trace ID

Service name

Very useful for debugging.

14. Recovery Flow

After issue fixed:

None

Read messages from DLQ

↓

Correct problem

↓

Replay to main queue

↓

Reprocess

15. Monitoring DLQ

DLQ should never be ignored.

Track:

- DLQ size
- Growth rate
- Oldest message age
- Failure reason categories
- Retry success %

Alert if sudden spike.

16. Real Examples

Payment Gateway

Payment callback failed:

→ store in DLQ

Inventory Update

Warehouse API timeout:

→ retry later from DLQ

SMS Provider Down

OTP send failed:

→ move to DLQ and retry

17. Interview Answer

Q: How would you handle failed third-party API writes?

Answer:

I'd first retry with exponential backoff. If retries fail, I'd move the request to a Dead Letter Queue. A retry worker or operator can later inspect and replay it. For production scale, I'd use Kafka/SQS DLQ rather than local memory because it's durable and scalable.

18. Best Practices

- Retry before DLQ
- Separate temporary vs permanent failures
- Add metadata to failed messages
- Monitor DLQ continuously
- Set replay tools
- Ensure idempotent reprocessing
- Alert on DLQ spikes

19. Short Revision Notes

None

DLQ = Queue for failed messages

Flow:

Process → Retry → Still Fail → DLQ

Storage Options:

1. Drop request
2. Local queue
3. Kafka/SQS (best)

Use Cases:

Payments, Emails, APIs, Orders

Need:

Monitoring + Replay + Idempotency